

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.М71-мм

Разработка сервиса для работы с учебными практиками

Каргин Глеб Павлович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
старший преподаватель кафедры системного программирования, к.т.н. Ю. В. Литвинов

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор	6
2.1. Обзор существующих решений	6
2.2. Обзор применяемых технологий	8
2.3. Анализ предыдущей версии системы	10
3. Проектирование	13
3.1. Архитектура системы	13
3.2. Схема взаимодействия сервисов	15
4. Реализация	16
4.1. Сервис нотификации	16
4.2. Система логирования действий пользователей	17
4.3. Механизмы гарантированной доставки сообщений	18
4.4. Сервис для анализа документов с использованием LLM .	19
5. Заключение	20
Список литературы	21

Введение

В СПбГУ студенты математико-механического факультета, начиная со второго курса, проходят учебные практики и выполняют выпускные квалификационные работы (ВКР), направленные на углубление теоретических знаний и развитие профессиональных навыков. В этот процесс вовлечены несколько участников, каждый из которых играет важную роль. Студент отвечает за выполнение практики или ВКР, выбирает тему из предложенных направлений и согласовывает её с научным руководителем и, при необходимости, с консультантом. Научный руководитель — преподаватель, курирующий выполнение работы, а также помогающий в постановке задач. Консультант — специалист, обладающий узкой экспертизой в выбранной теме, который может быть как преподавателем СПбГУ, так и представителем внешней организации, и оказывает помощь по технической стороне работы. После завершения работы отчёт студента направляется на выборочное рецензирование, где рецензент оценивает соответствие требованиям, полноту и качество выполненного задания. За общую организацию и координацию процесса отвечает руководитель практики — сотрудник, отслеживающий соблюдение сроков и обеспечивающий взаимодействие между всеми участниками.

Однако процесс выбора тем, подачи отчётов и отслеживания выполнения практик остаётся трудоёмким и требует значительных усилий со стороны всех участников. В настоящее время на сайте кафедры системного программирования СПбГУ существует сервис, который призван автоматизировать часть этих процессов. Тем не менее, его функциональность имеет ряд существенных недостатков. Система сложна в поддержке, её интерфейс не всегда интуитивно понятен, а архитектура перегружена и малоадаптирована для эффективной работы. Кроме того, сервис интегрирован с основным сайтом кафедры, предназначенным в первую очередь для публикации статического информационного контента, а не для организации и управления учебными процессами. Попытка реализовать такую многоуровневую систему на статическом

сайте привела к трудностям как для разработчиков, так и для пользователей.

В связи с этим было принято решение создать отдельный сервис, независимый от основного сайта кафедры, который будет направлен исключительно на автоматизацию всех этапов учебной практики — от сбора и выбора тем до оценки результатов и размещения материалов успешных практик. Новый сервис предназначен для взаимодействия студентов, преподавателей и внешних экспертов на всех этапах процесса. Он включает сбор и проверку отчётов, а также контроль за выполнением задач.

Для реализации этой идеи разрабатывается специализированное веб-приложение с микросервисной архитектурой, что обеспечивает поддержку и возможность расширения функциональности системы для работы с учебными практиками СПбГУ. Разработка данного сервиса была начата в рамках выпускной квалификационной работы [8], однако масштаб проекта требует его дальнейших реализации и развития.

1. Постановка задачи

Целью работы является создание веб-приложения для управления процессом прохождения учебных практик. Для её выполнения были поставлены следующие задачи.

1. Разработать сервис уведомлений.
2. Реализовать систему логирования действий пользователей.
3. Внедрить механизмы гарантированной доставки сообщений через RabbitMQ.
4. Исправить имеющиеся недочеты системы.
5. Начать реализацию сервиса для анализа документов с использованием большой языковой модели (LLM).

2. Обзор

2.1. Обзор существующих решений

- **Сервис учебных практик кафедры системного программирования¹**. На момент начала разработки на кафедре эксплуатировалась действующая версия веб-сервиса для сопровождения учебных практик и выпускных квалификационных работ (Рис. 1). Система реализовывала базовую функциональность: ведение реестра тем, их распределение среди студентов, процесс рецензирования и сопровождения работ [9]. В ходе анализа были выявлены архитектурные и функциональные ограничения, затрудняющие развитие системы: монолитная архитектура, высокая связанность компонентов, отсутствие механизмов архивации данных по завершении учебного периода, а также сложности интеграции с другими кафедральными сервисами.

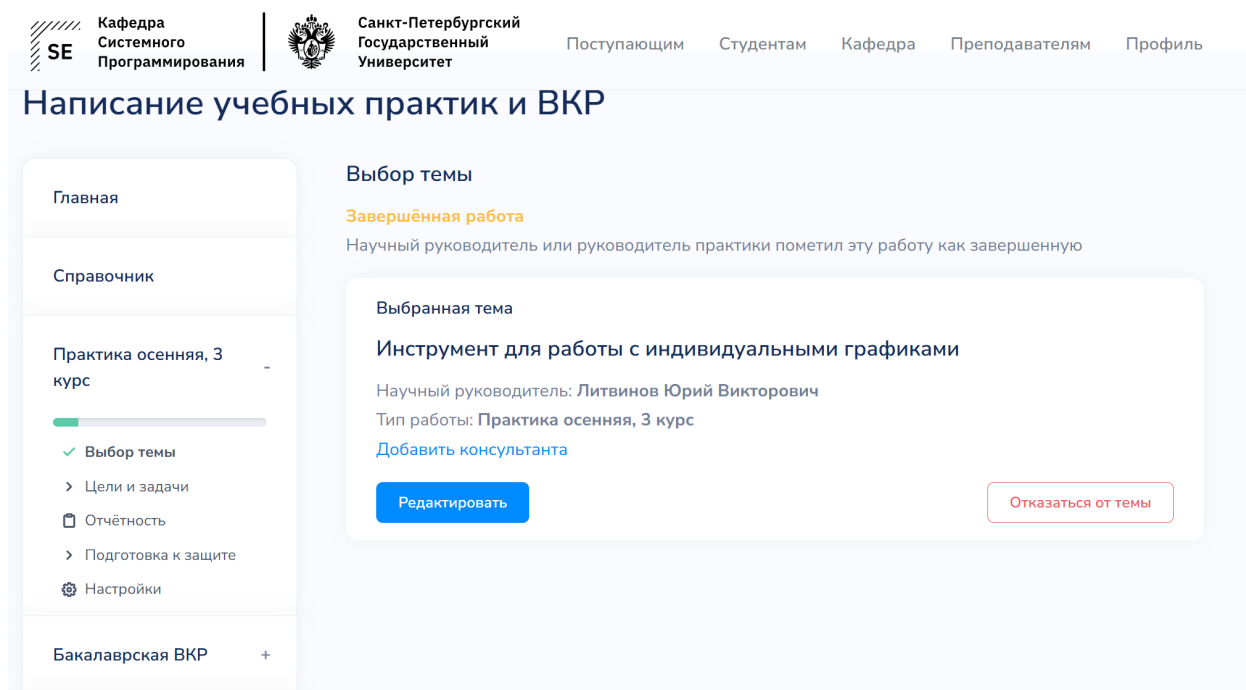


Рис. 1: Интерфейс действующего сервиса учебных практик

¹Сервис учебных практик и ВКР. URL: <https://se.math.spbu.ru/practice> (дата обращения: 06.07.2026)

- **Системы управления проектами (Jira)².** Инструменты управления проектами предлагают развитые механизмы управления задачами: создание тикетов, назначение исполнителей, контроль сроков, комментирование, вложение файлов. Их адаптация для образовательных процессов требует существенной доработки и не обеспечивает готовых решений для учёта академических ролей (студент, научный руководитель), автоматизации распределения тем.
- **Системы контроля версий (GitHub)³.** В образовательном контексте системы контроля версий могут использоваться для сдачи текстов работ через механизм pull request, однако данные системы не содержат встроенных функций для администрирования учебного процесса, управления темами и взаимодействия с преподавателями вне контекста репозитория.
- **Облачные офисные платформы (Яндекс)⁴.** Наборы сервисов для совместной работы (документы, календари, дисковые хранилища) предоставляют базовые возможности для коммуникации и обмена материалами между студентами и научными руководителями. Их функциональность не покрывает потребности в структурированном управлении практиками: отсутствуют механизмы автоматического распределения, контроля этапов выполнения, формирования отчётности и интеграции с системами вуза.

Проведённый анализ показал, что существующие решения либо не обладают необходимой специализированной функциональностью, либо требуют значительных доработок для адаптации к процессу управления учебными практиками. Это привело к необходимости разработки новой системы с учётом специфических требований кафедры.

²Jira. URL: <https://www.atlassian.com/ru/software/jira> (дата обращения: 06.07.2026)

³GitHub. URL: <https://github.com> (дата обращения: 06.07.2026)

⁴Яндекс.Календарь. URL: <https://calendar.yandex.ru> (дата обращения: 06.07.2026)

2.2. Обзор применяемых технологий

2.2.1. Платформа .NET и ASP.NET Core

В качестве базовой платформы выбрана .NET [5], что соответствует технологическим стандартам, принятым на кафедре, и обеспечивает поддержку большого числа студентов и преподавателей, знакомых с данной экосистемой. Язык программирования C# используется в связи с его распространённостью в университетских проектах. Для реализации веб-сервисов применён фреймворк ASP.NET Core [1] с использованием Minimal API, что позволяет создавать конечные точки с автоматической сериализацией данных в JSON, минимизируя количество шаблонного кода по сравнению с классическим подходом на контроллерах.

2.2.2. Микросервисная архитектура и YARP

Архитектура системы построена на принципах микросервисов, что обеспечивает декомпозицию функциональности на независимые, слабосвязанные компоненты. Для маршрутизации HTTP-запросов между микросервисами используется YARP (Yet Another Reverse Proxy) [7] — обратный прокси, интегрируемый в среду .NET. Его применение позволяет централизованно управлять правилами маршрутизации, реализовывать балансировку нагрузки и динамически конфигурировать взаимодействие сервисов.

2.2.3. Entity Framework Core и PostgreSQL

Для объектно-реляционного отображения используется Entity Framework Core [4]. Данный фреймворк абстрагирует работу с реляционной базой данных через сущности (`DbSet<TEntity>`) и контекст данных (`DbContext`), что упрощает выполнение CRUD-операций и управление схемой данных. В качестве системы управления базами данных выбрана PostgreSQL [6]. Выбор обоснован необходимостью гарантировать целостность данных при операциях, затрагивающих несколько взаимосвязанных сущностей (студенты, учебные группы, те-

мы практик, руководители). Кроме того, поддержка сложных запросов с множественными объединениями необходима для построения отчётности и фильтрации данных (поиск доступных тем, формирование списков студентов по группам).

2.2.4. RabbitMQ

Для организации асинхронного обмена сообщениями между микросервисами внедрён брокер RabbitMQ. Он реализует модель очередей сообщений и используется для синхронизации данных, запуска фоновых задач и обеспечения согласованности в распределённой системе. Выбор RabbitMQ обусловлен его совместимостью с .NET, наличием механизмов гарантированной доставки и гибкостью маршрутизации.

2.2.5. Docker и Docker Compose

Контейнеризация компонентов системы выполнена с использованием Docker [2], что создать изолированные контейнеры для каждого сервиса, их баз данных, а также обратного прокси, автоматизируя их запуск и настройку. Оркестрация многоконтейнерного развёртывания осуществляется инструментом Docker Compose [3], который упрощает определение зависимостей, настройку сетевого взаимодействия и управление жизненным циклом приложения в средах разработки и тестирования.

2.2.6. Frontend: React и Vite

Пользовательский интерфейс реализован на библиотеке React с использованием TypeScript для обеспечения типизации. Сборка пользовательского интерфейса приложения осуществляется с помощью инструмента Vite, который предоставляет быструю разработку с Hot Module Replacement (HMR) в режиме разработки и оптимизированную сборку для режима production.

2.3. Анализ предыдущей версии системы

На начальном этапе эксплуатации версии MVP (Minimum Viable Product) ⁵ был выявлен ряд архитектурных недочетов и технических проблем, требующих устранения для обеспечения корректной работы системы в промышленной эксплуатации.

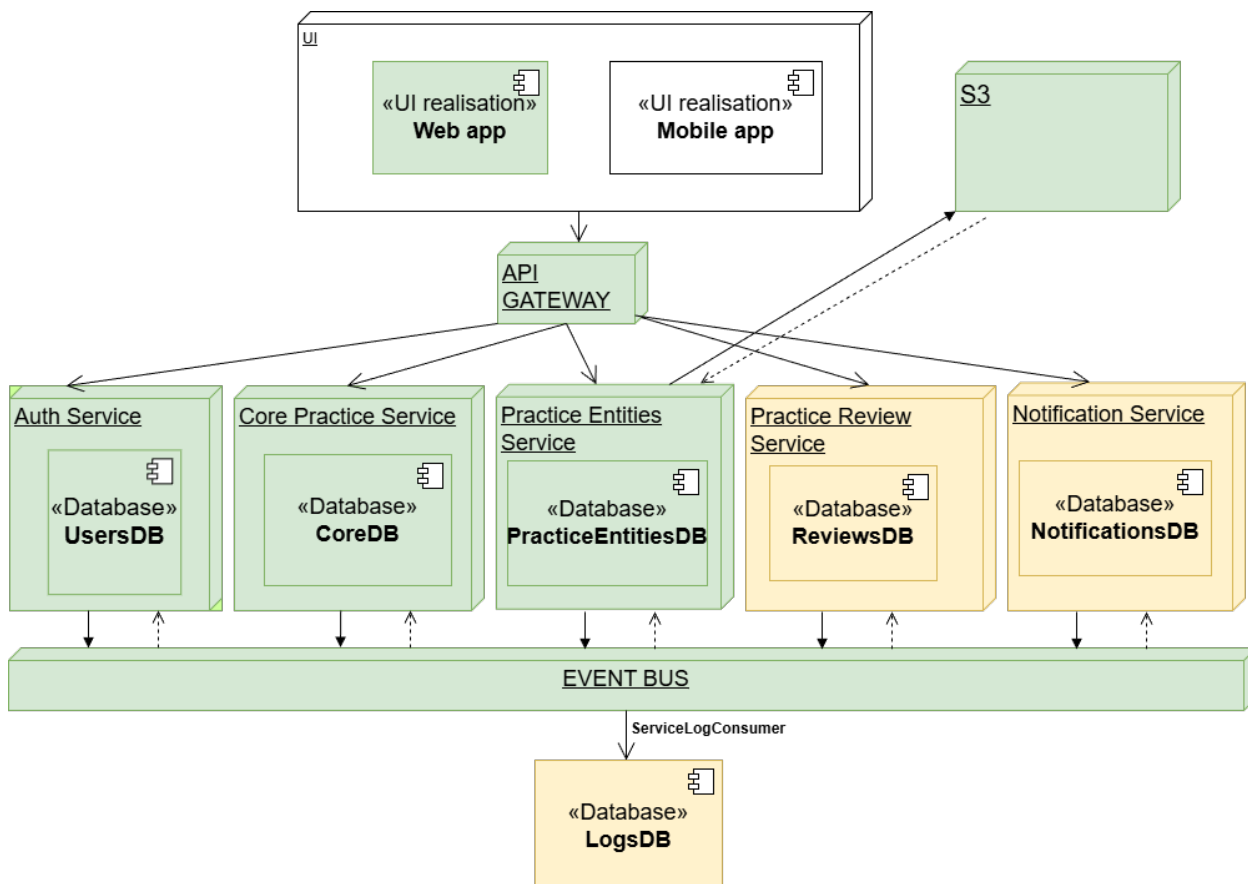


Рис. 2: Диаграмма предыдущей версии. Зеленым цветом выделены реализованные компоненты

Анализ существующей реализации показал, что, несмотря на работоспособность, система нуждается в ряде доработок архитектурного характера.

⁵MVP. URL: https://ru.wikipedia.org/wiki/Минимально_жизнеспособный_продукт (дата обращения: 06.07.2026)

2.3.1. Асинхронное взаимодействие микросервисов

В текущей реализации отсутствовала надежность механизма асинхронного обмена сообщениями между компонентами системы. Это приводило к нарушению согласованности данных при недоступности сервисов или временных сбоях в сети. Отсутствие подтверждений доставки и механизмов повторных попыток обработки сообщений создавало риск потери важных событий. Для решения данной проблемы требовалась корректная настройка брокера сообщений, обеспечивающего гарантированную доставку событий между микросервисами.

2.3.2. Сервис нотификации

В текущей версии отсутствовал централизованный сервис, отвечающий за отправку уведомлений пользователям системы. Такие функции, как подтверждение регистрации, восстановление пароля или оповещение об изменении статуса практики, не были реализованы в полном объеме. Требуется создание отдельного микросервиса, который бы обеспечивал отправку email-уведомлений и интеграцию с брокером сообщений для асинхронной обработки запросов.

2.3.3. Логирование

Отсутствие единой системы сбора и хранения информации о действиях пользователей и системных событиях затрудняет анализ инцидентов и аудит безопасности. Для обеспечения возможности отслеживания критических операций (вход в систему, изменение ролей, регистрация новых пользователей) необходимо внедрение централизованного сервиса логирования с выделенным хранилищем.

2.3.4. Автоматизация проверки документов

Перспективным направлением развития является внедрение интеллектуальных механизмов анализа отчетов, презентаций и отзывов. Автоматизация этого процесса с использованием технологий искусственного

интеллекта позволит снизить нагрузку на преподавателей и повысить качество обратной связи для студентов. Для реализации такого функционала требуется создание отдельного сервиса, интегрирующегося с существующей инфраструктурой и использующего большие языковые модели для анализа содержимого документов.

Таким образом, анализ текущего состояния системы позволил определить ключевые направления ее развития, которые будут рассмотрены в следующих разделах данной работы.

3. Проектирование

На этапе проектирования были определены архитектурный стиль, декомпозиция системы на микросервисы, разработаны схемы баз данных и созданы прототипы пользовательского интерфейса.

3.1. Архитектура системы

Для обеспечения масштабируемости и возможности независимой разработки модулей была выбрана микросервисная архитектура. Система предполагает наличие веб-клиента и, в перспективе, мобильного приложения. Входной точкой для всех запросов служит шлюз (API Gateway), который выполняет маршрутизацию к функциональным сервисам.

На основе анализа, проведенного в предыдущем разделе, в архитектуру системы были внесены изменения, направленные на устранение выявленных недостатков. В состав системы добавлены следующие компоненты:

- **Сервис нотификации (Notification Service)** — обеспечивает централизованную отправку email-уведомлений, включая подтверждение регистрации, восстановление пароля и оповещения об изменениях в практиках.
- **Сервис логирования (Audit Service)** — осуществляет сбор и хранение информации о действиях пользователей и системных событиях. Данный компонент позволяет отслеживать критически важные операции (вход в систему, изменение ролей, регистрация новых пользователей) и фиксировать возникающие ошибки, что повышает прозрачность системы и упрощает анализ инцидентов.
- **Сервис анализа документов (LLM Service)** — обеспечивает автоматическую проверку загружаемых документов (отчетов, презентаций, отзывов) с использованием больших языковых моделей.

Для организации надежного асинхронного взаимодействия между

компонентами используется брокер сообщений RabbitMQ, обеспечивающий гарантированную доставку событий.

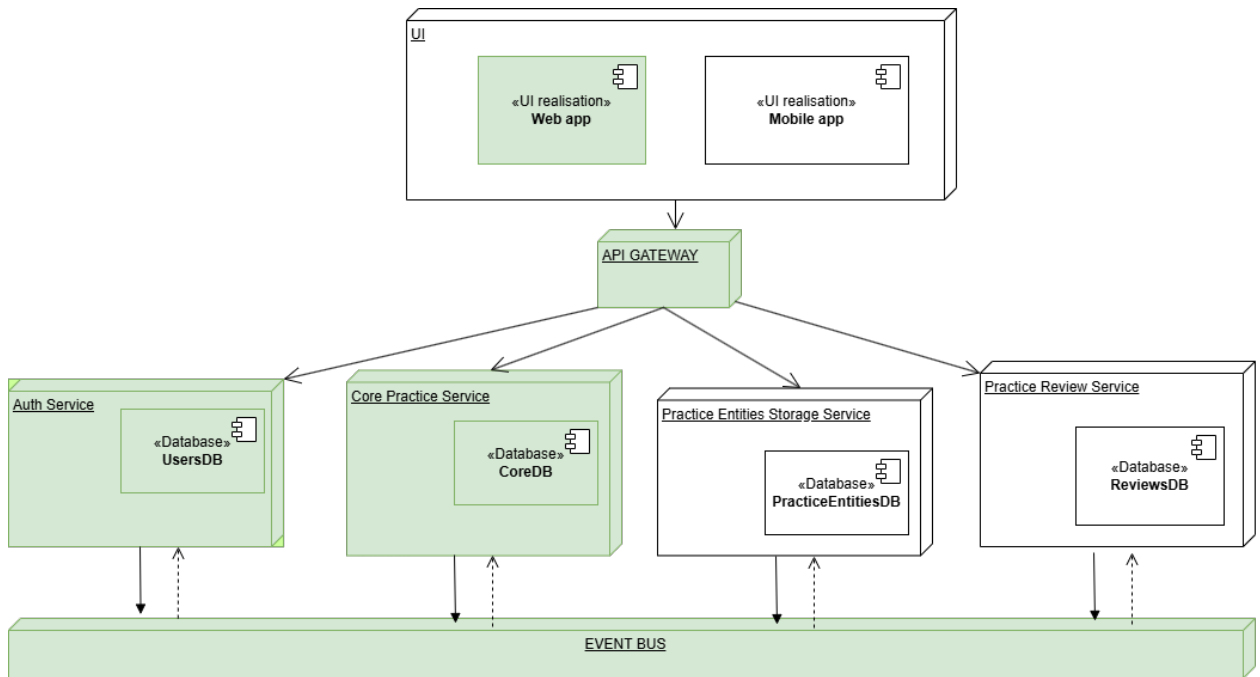


Рис. 3: Архитектура системы на начальном этапе

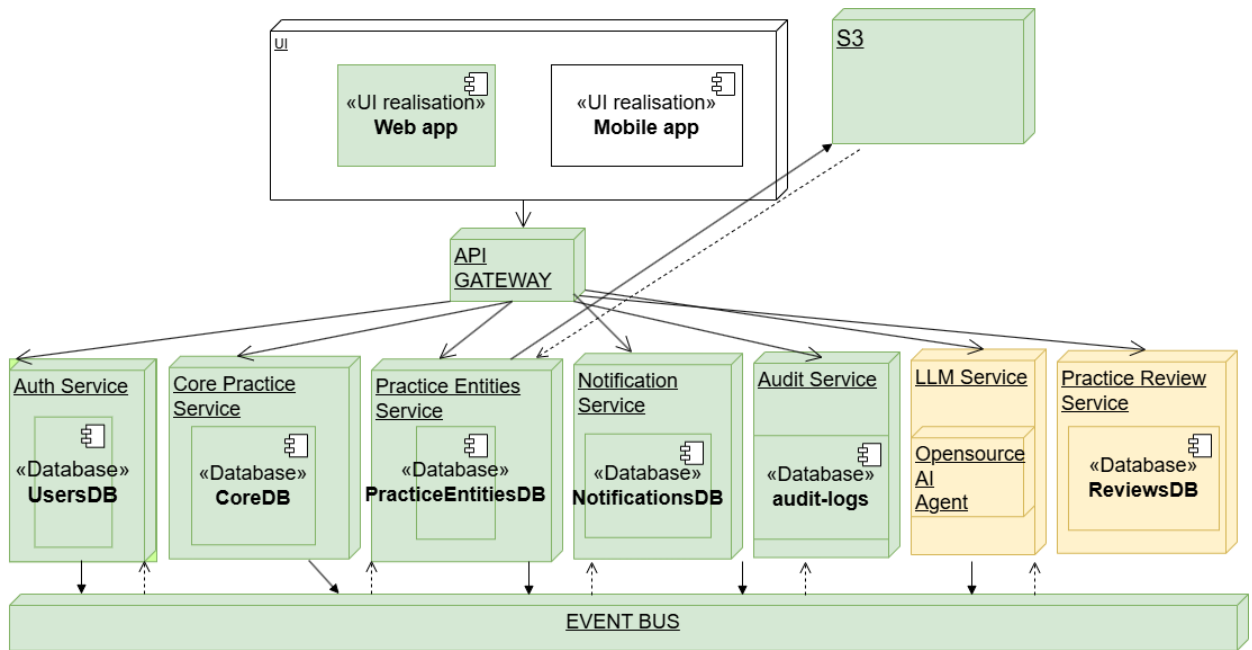


Рис. 4: Обновленная компонентная диаграмма системы

3.2. Схема взаимодействия сервисов

Взаимодействие между компонентами системы организовано следующим образом. Клиентские запросы поступают на API Gateway, который на основе маршрутизации перенаправляет их к соответствующим сервисам. Аутентификация и авторизация выполняются в Auth Service, который выдает JWT-токены, используемые для защиты эндпоинтов.

При изменении состояния сущностей (создание пользователя, назначение роли, изменение практики) сервисы публикуют события в RabbitMQ. Это позволяет другим компонентам системы реагировать на изменения асинхронно. Например, при изменении роли пользователя публикуется событие, которое обрабатывается Core Service для синхронизации профиля, а также Notification Service для отправки уведомления.

Сервис аудита подписан на все значимые события и сохраняет их в отдельном хранилище для последующего анализа. Такой подход позволяет централизованно логировать действия пользователей без внесения изменений в бизнес-логику сервисов.

4. Реализация

В рамках данной работы были реализованы следующие компоненты системы.

4.1. Сервис нотификации

Одной из ключевых задач являлась разработка микросервиса для централизованной отправки email-уведомлений. Данный компонент отвечает за оповещение пользователей о различных событиях в системе: подтверждение регистрации, восстановление пароля, изменение статуса практик и другие уведомления.

Сервис нотификации построен как отдельный микросервис, взаимодействующий с остальными компонентами системы через брокер сообщений RabbitMQ. Такой подход позволяет обрабатывать запросы на отправку писем асинхронно, не блокируя выполнение основных бизнес-операций.

Архитектура сервиса включает потребителей сообщений (Consumers) — каждый тип уведомления обрабатывается отдельным потребителем, подписанным на соответствующую очередь в RabbitMQ. Это позволяет изолировать обработку различных сценариев и упрощает масштабирование. Кроме того, в состав сервиса входит Email Service, отвечающий за формирование и отправку электронных писем через SMTP-сервер с поддержкой HTML-форматирования для создания структурированных уведомлений.

В рамках реализации сервиса были созданы потребители для следующих типов событий:

- `EmailConfirmationConsumer` — отправка ссылки для подтверждения электронной почты при регистрации;
- `PasswordResetRequestedConsumer` — отправка ссылки для восстановления пароля;
- `PracticeUpdatedConsumer` — уведомление об изменении статуса

практики;

- `FeedbackSubmittedConsumer` — оповещение о новом отзыве;
- `MessageSentConsumer` — уведомление о новом сообщении;
- `TextWorkSubmittedConsumer` — оповещение о загрузке текстовой работы;
- `TextWorkCommentAddedConsumer` — уведомление о новом комментарии к текстовой работе;
- `PresentationSubmittedConsumer` — уведомление о загрузке презентации;
- `PresentationCommentAddedConsumer` — уведомление о новом комментарии к презентации;
- `ReportSubmittedConsumer` — оповещение о загрузке отчета;
- `ReportCommentAddedConsumer` — уведомление о новом комментарии к отчету;
- `ThemeArchivedConsumer` — уведомление об архивации темы;
- `GoalsAndTasksUpdatedConsumer` — оповещение об обновлении целей и задач практики.

Каждый потребитель обрабатывает входящее событие и формирует соответствующее письмо, которое затем отправляется через SMTP-клиент.

4.2. Система логирования действий пользователей

Для обеспечения мониторинга и аудита действий пользователей был реализован отдельный микросервис `Audit Service`, выполняющий централизованный сбор и хранение информации о событиях, происходящих в системе.

Сбор событий организован таким образом, что каждый значимый компонент системы публикует события в RabbitMQ при выполнении критических операций: вход в систему, изменение ролей, регистрация новых пользователей, создание и редактирование практик. Обработка событий осуществляется потребителем `AuditConsumer`, который принимает события из очереди и сохраняет их в базе данных. Для каждого события фиксируется идентификатор пользователя, его email, роли, тип выполненного действия, идентификатор сущности, дополнительные параметры, статус выполнения (успех/ошибка) и время события. Хранение аудиторских записей реализовано на базе MongoDB, что позволяет эффективно хранить документы с различной структурой деталей события и обеспечивает высокую скорость записи.

Для обеспечения быстрого доступа к данным в коллекции `audit_logs` созданы индексы по полям `Action` (тип действия) и `UserEmail` (пользователь). Это позволяет выполнять фильтрацию и поиск записей с минимальной задержкой.

Сервис аудита предоставляет конечные точки для получения информации, доступ к которым ограничен политикой `AdminOnly`, что обеспечивает просмотр логов только администраторами системы.

4.3. Механизмы гарантированной доставки сообщений

Для обеспечения надежности асинхронного взаимодействия между микросервисами в RabbitMQ были внедрены механизмы, повышающие отказоустойчивость системы. Для каждого потребителя настроены политики повторных попыток (`Retry Policies`) — например, при сбое SMTP-сервера в сервисе нотификации сообщение повторно обрабатывается через заданные интервалы (1, 2, 5 секунд) до успешного выполнения или исчерпания лимита попыток. Очереди разделены по типам событий, что позволяет гибко настраивать приоритеты обработки: критически важные уведомления (сброс пароля) обрабатываются с более высоким приоритетом и меньшим количеством параллельных потреби-

телей, чем менее значимые события (фидбеки). Кроме того, настроены параметры Heartbeat и таймауты подключения, обеспечивающие автоматическое переподключение при потере связи с брокером.

Внедрение указанных механизмов позволило повысить надежность системы и гарантировать обработку всех событий даже при временных сбоях отдельных компонентов.

4.4. Сервис для анализа документов с использованием LLM

Для автоматизации проверки документов (отчетов, презентаций, отзывов) была начата разработка отдельного микросервиса, использующего большие языковые модели (LLM). Данный компонент предназначен для предоставления пользователям автоматической обратной связи по качеству оформления и содержанию загружаемых документов.

В рамках начального этапа реализации были созданы базовые агенты для анализа отчетов и проверки орфографии. Дальнейшее развитие предполагает расширение набора агентов, внедрение в процесс загрузки документов и улучшение качества анализа.

5. Заключение

В ходе выполнения данной работы были получены следующие результаты.

- Обновлено архитектура системы с добавлением новых компонентов.
- Разработан сервис нотификации для централизованной отправки email-уведомлений.
- Реализована система логирования действий пользователей.
- Внедрены механизмы гарантированной доставки сообщений через RabbitMQ.
- Начата реализация сервиса для анализа документов с использованием LLM.

Код доступен в репозитории на GitHub⁶.

⁶PracticesService. URL: <https://github.com/spbu-se/PracticesService> (дата обращения: 2026-07-06). [Имя аккаунта: Belgrak]

Список литературы

- [1] ASP.NET Core. — URL: <https://learn.microsoft.com/ru-ru/aspnet/core/?view=aspnetcore-8.0> (дата обращения: 2026-07-06).
- [2] Docker. — URL: <https://www.docker.com> (дата обращения: 2026-07-06).
- [3] Docker Compose. — URL: <https://docs.docker.com/compose> (дата обращения: 2026-07-06).
- [4] Entity Framework. — URL: <https://learn.microsoft.com/ru-ru/ef> (дата обращения: 2026-07-06).
- [5] .NET. — URL: <https://dotnet.microsoft.com/ru-ru> (дата обращения: 2026-07-06).
- [6] PostgreSQL. — URL: <https://www.postgresql.org> (дата обращения: 2026-07-06).
- [7] YARP. — URL: <https://microsoft.github.io/reverse-proxy> (дата обращения: 2026-07-06).
- [8] Каргин Г. П. Разработка сервиса для работы с учебными практиками. — URL: https://se.math.spbu.ru/thesis/texts/Kargin_Gleb_Pavlovich_Bachelor_Thesis_2025_text.pdf (дата обращения: 6 июля 2026 г.).
- [9] Слугин А. Н. Разработка веб-сервиса для написания учебных практик и выпускных квалификационных работ. — URL: https://se.math.spbu.ru/thesis_download?thesis_id=1288 (дата обращения: 6 июля 2026 г.).